IT 112: Web Systems & Technologies

BSIT-2B 2nd Sem

S.Y. 2025-2026

Final Project Proposal

Phase 5 Documentation for the Quick Reserve :Online Bus & Travel Booking System A Final Project by the <Tech_ Vanguard />(Week 14)

Submitted By:

Nathaniel O. Raynada

Anna Mae P. Broma

Camille T. Bercilla

Mariel O. Odoño

John Edmar L. Bongcodin

Justin James Dasallas

Submitted To:

Dr. Guillermo V. Red, Jr, DIT

April 2026

I. Overview: Data Retrieval and Display

The QuickReserve system focuses on retrieving and displaying data on the frontend to provide a dynamic and interactive user experience. Originating in Albay, the system highlights local routes such as Legazpi City, Naga City, Sorsogon City, Daet, Ligao City, and Polangui, as shown in the interface above. It provides specific travel routes along with corresponding estimated travel times and fare ranges, allowing users to easily choose their desired destination.

Through the integration of frontend and backend components, the system is able to present real-time information such as users, bookings, and trips. This allows both users and administrators to access updated data directly on the interface without the need for manual refresh or input.

The implementation uses JavaScript to handle data retrieval from the backend through API requests. Methods such as fetch() are utilized to obtain data in JSON format, which is then processed and displayed using DOM manipulation techniques. Elements such as tables, cards, and lists are dynamically generated to organize the data in a clear and structured way. This ensures that the information presented is both readable and responsive to user interactions.

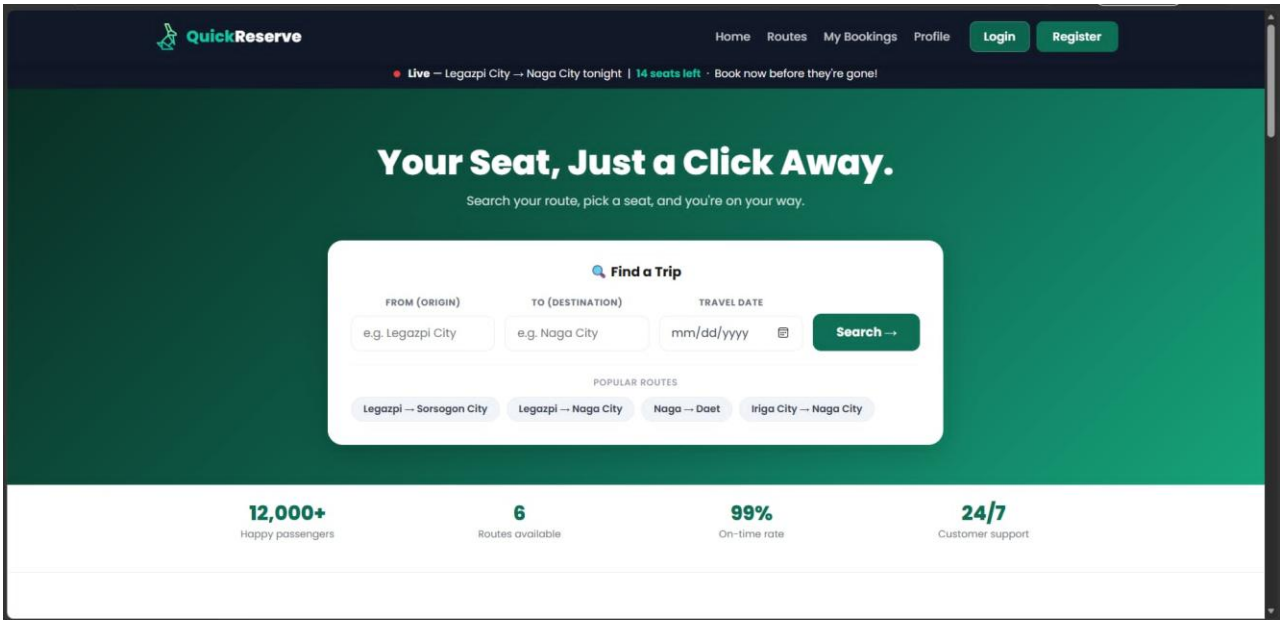
Additionally, the system emphasizes the importance of designing user-friendly interfaces that can efficiently handle and present large amounts of data. Features such as search functionality and simple pagination are implemented to improve navigation and usability. These features allow users to easily filter and locate specific information within the system.

Finally, the data retrieval and display process plays a crucial role in transforming the QuickReserve system into a fully functional web application. It demonstrates how data can be efficiently fetched, processed, and presented to users in real time, ensuring accuracy, accessibility, and an improved overall user experience.

II. Display Pages (Screenshots)

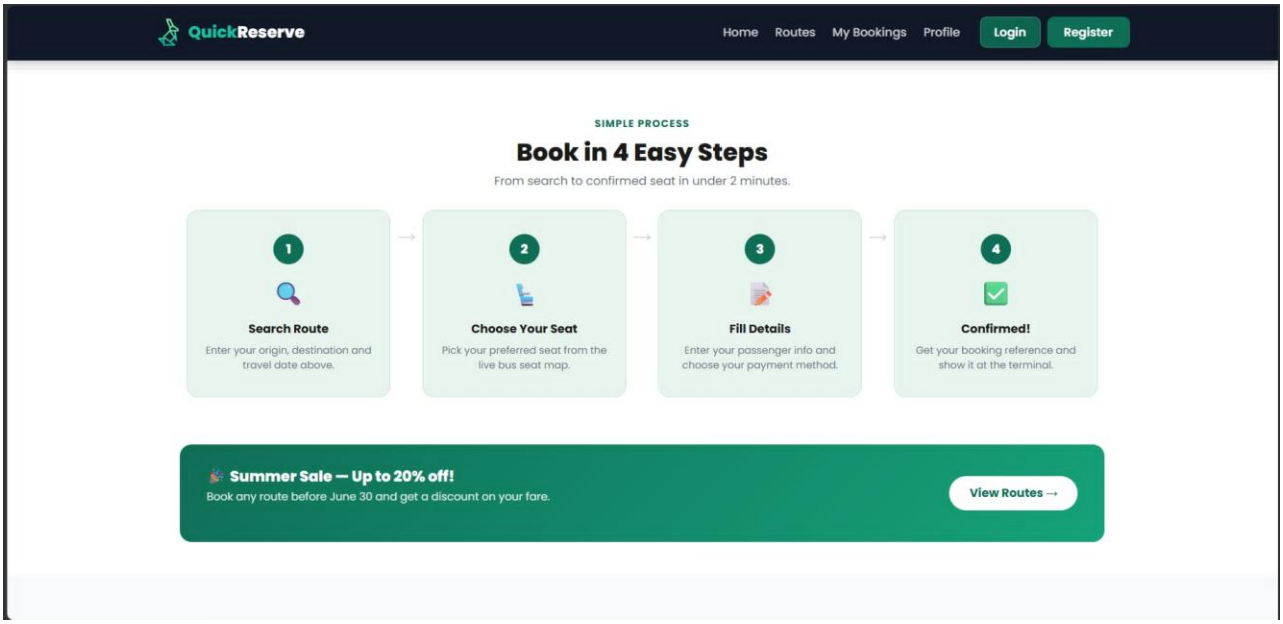
Homepage Search Interface

This screenshot displays the main homepage of the QuickReserve system, featuring the trip search interface. Users can input their origin, destination, and travel date to find available routes. The layout is designed to be simple and user-friendly, with clearly labeled fields and a prominent search button to initiate the process. Additional elements such as popular route suggestions and system statistics (e.g., number of passengers, routes available, and service reliability) are included to enhance user engagement and trust. This page serves as the entry point of the system, allowing users to easily begin their booking journey.



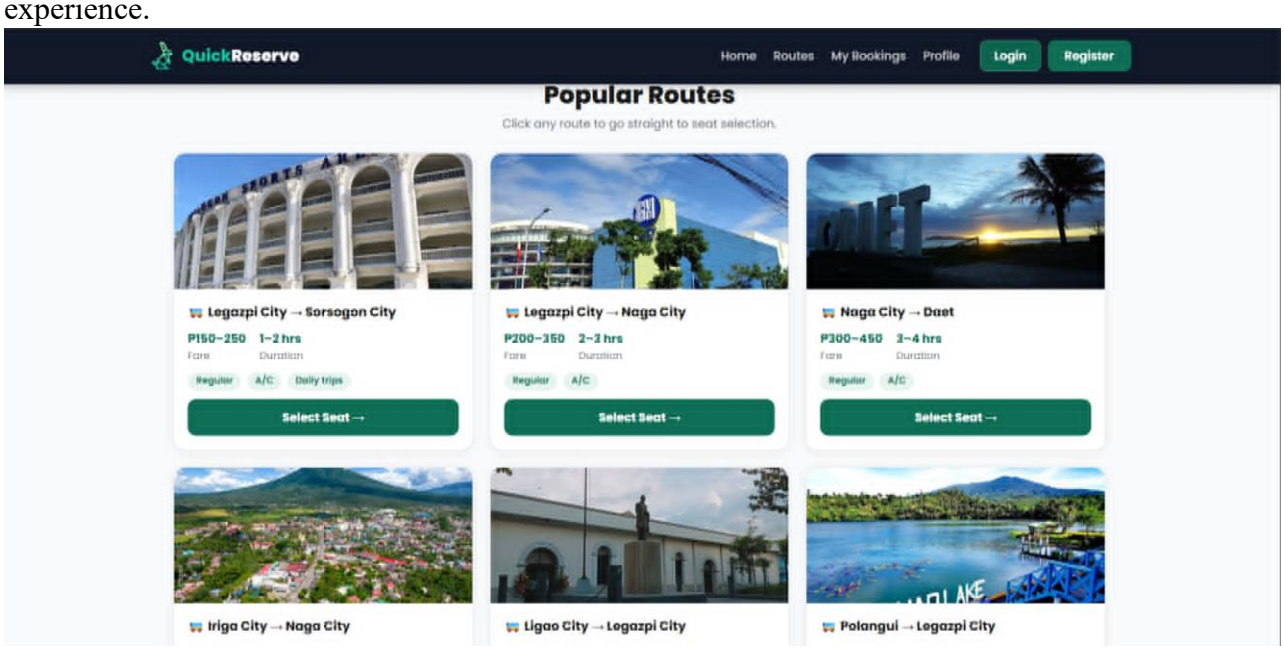
Booking Process Overview

The booking process overview of the QuickReserve system, which guides users through the steps required to complete a reservation. The interface highlights four main steps: searching for a route, selecting a seat, filling in passenger details, and confirming the booking. Each step is visually represented using icons and short descriptions to make the process easy to follow. This section enhances usability by providing clear instructions and setting user expectations during the booking process. It helps ensure that users can complete their transactions smoothly and without confusion.



Popular Routes

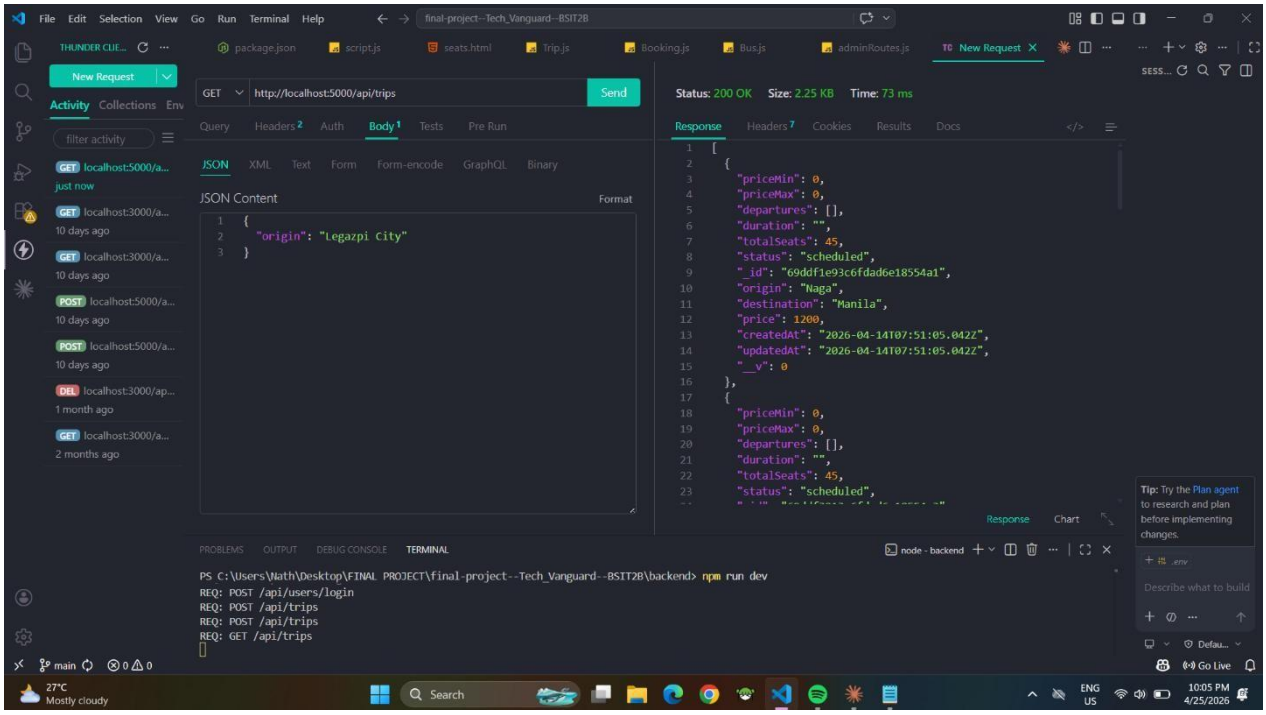
The “Popular Routes” section of the QuickReserve system, where available travel routes are displayed in a card-based layout. Each card presents key information such as origin and destination, fare range, estimated travel duration, and available features like air-conditioned buses and daily trips. The interface is designed to be visually organized and easy to understand, allowing users to quickly compare different routes. The “Select Seat” button provides direct interaction, enabling users to proceed with booking by choosing their preferred seat. This page demonstrates how travel data is clearly presented to support efficient decision-making and improve user experience.



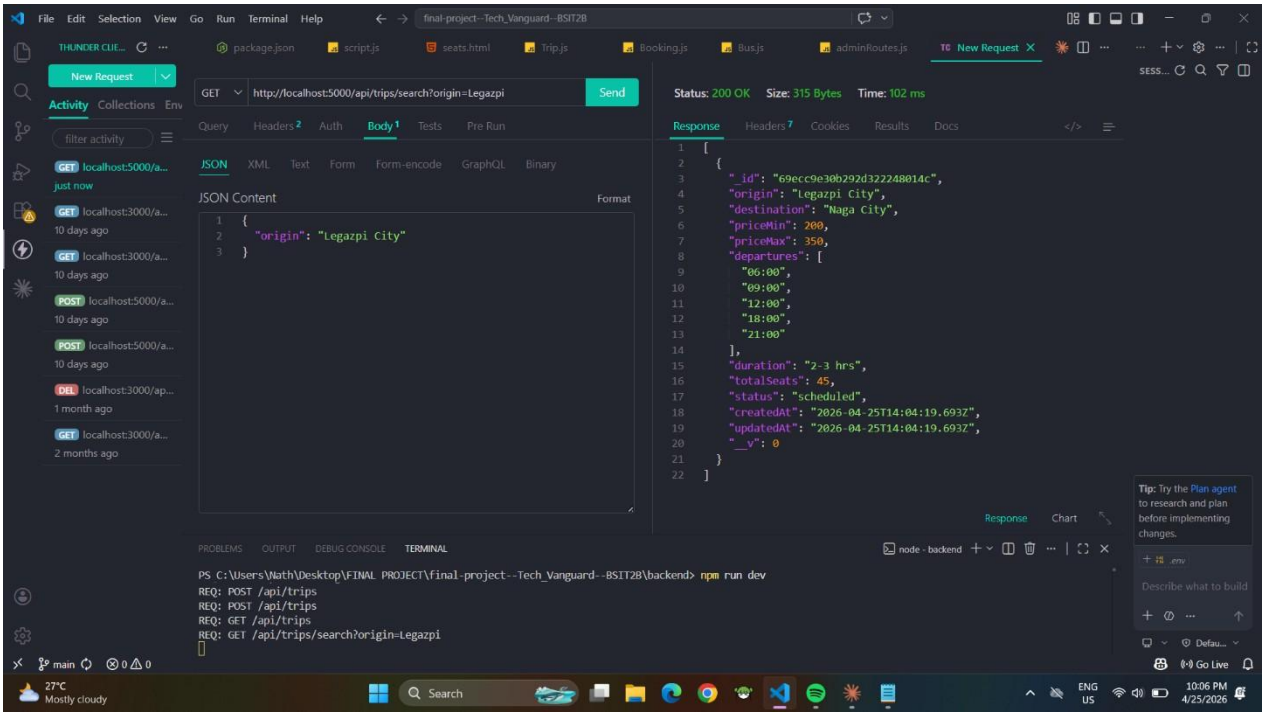
III. Backend API Response (Sample Output)

This shows a GET request executed in Postman to retrieve data from the backend API. The request includes the endpoint URL and uses the GET method to access stored data. The response is displayed in JSON format, containing structured information such as users, bookings, or trips. This confirms that the backend is functioning properly and is able to return accurate and organized data. The retrieved data is essential for frontend integration, as it is used to dynamically display information within the system’s interface.

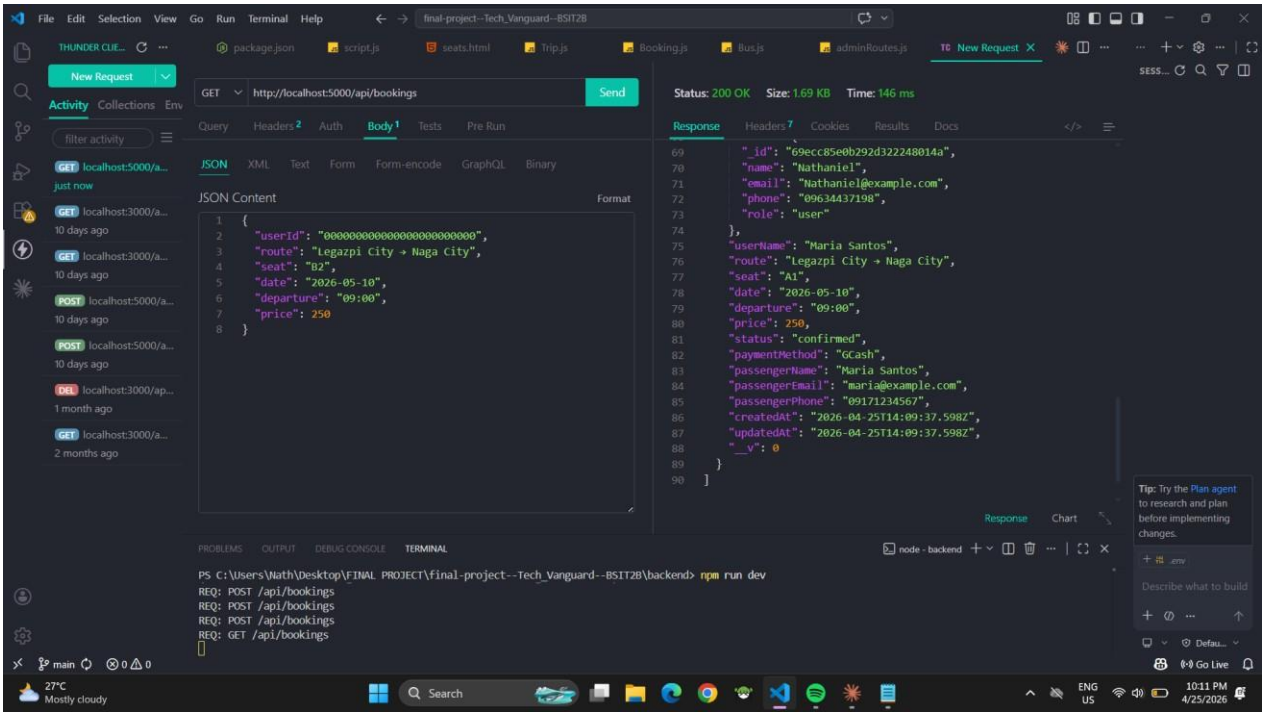
1. Get All Trips / GET request to retrieve all trips. It displays available routes.



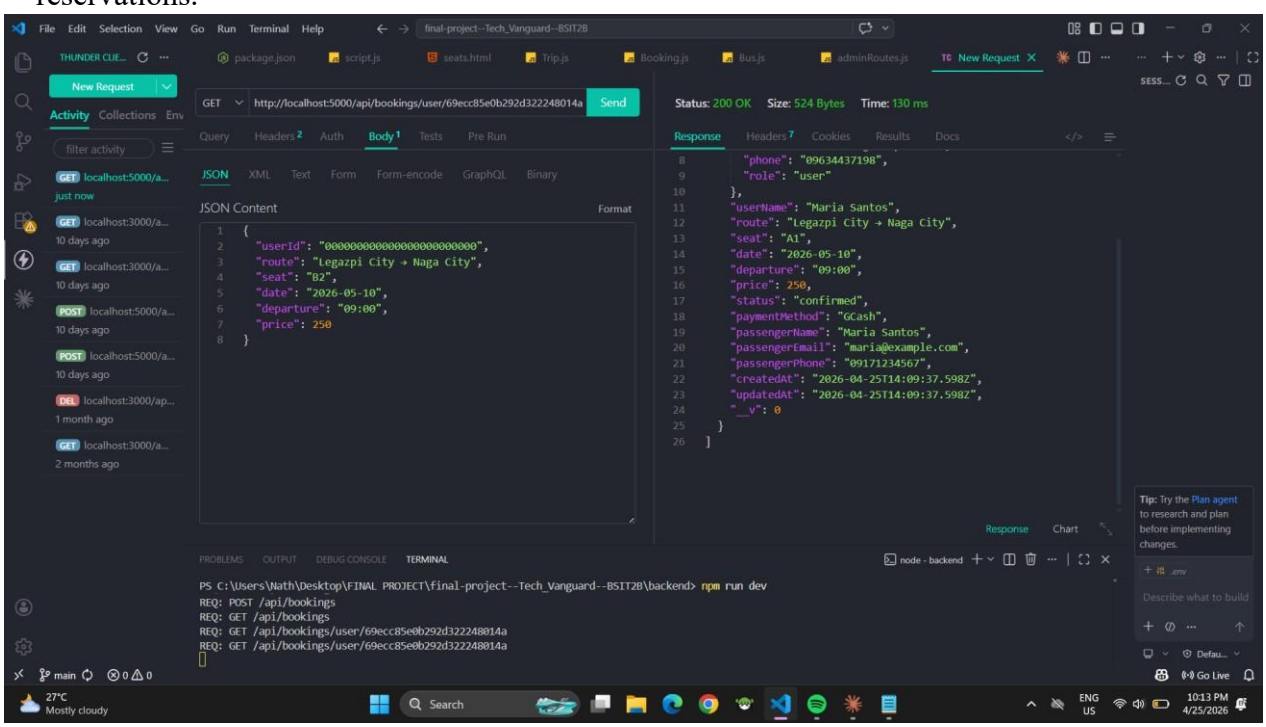
2. Search Trips / GET request to find trips by origin. It filters results.



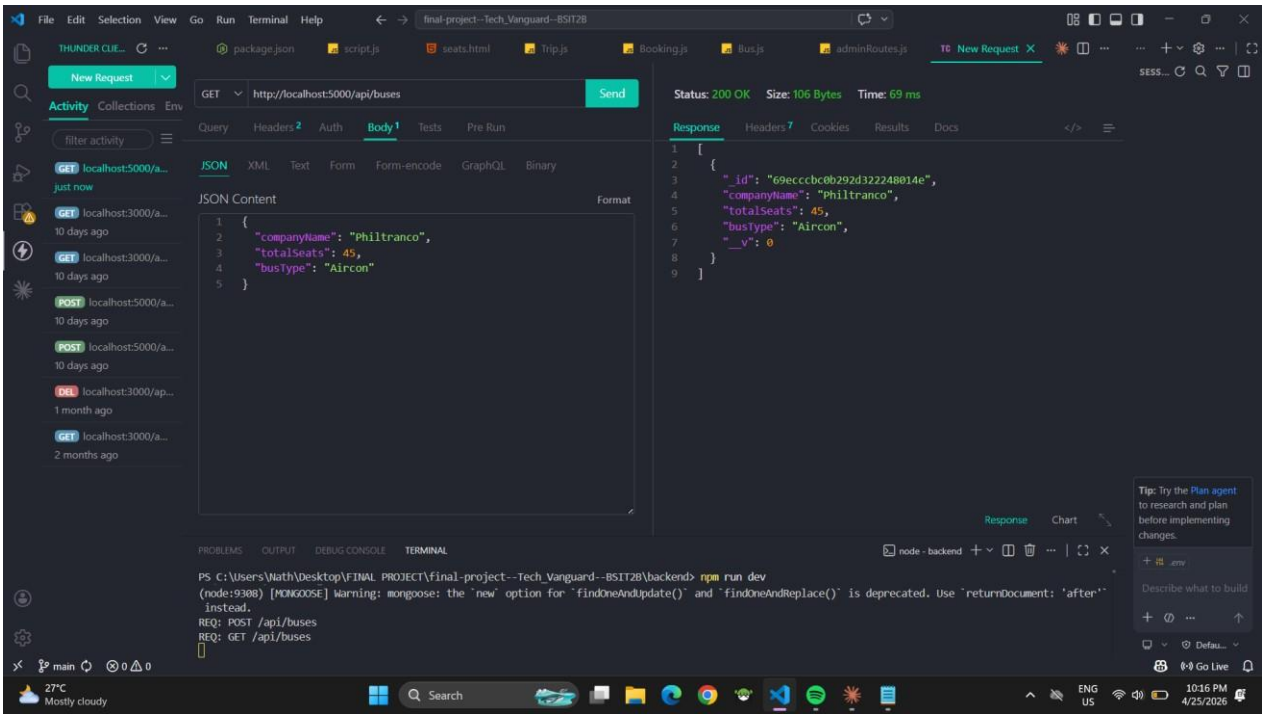
3. **Get All Bookings/** GET request to retrieve all bookings. It may include user info.



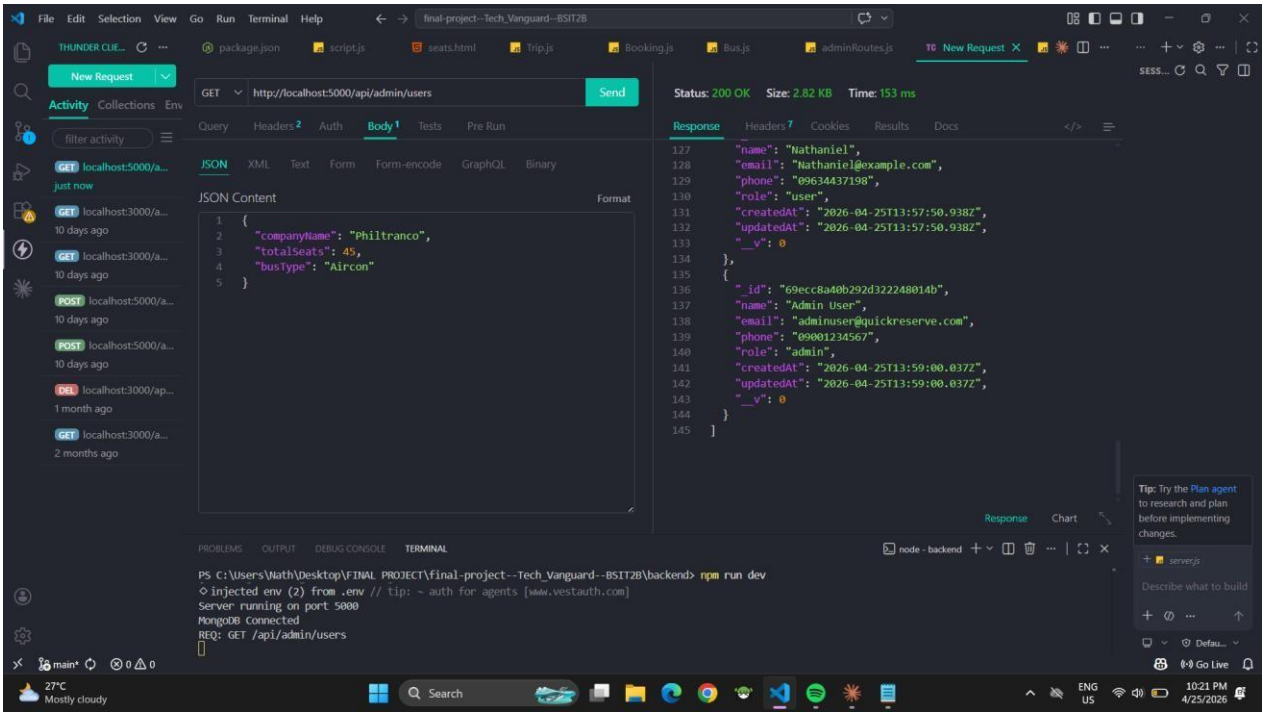
4. **Get Bookings by User /** GET request to retrieve bookings of a specific user. It shows user reservations.



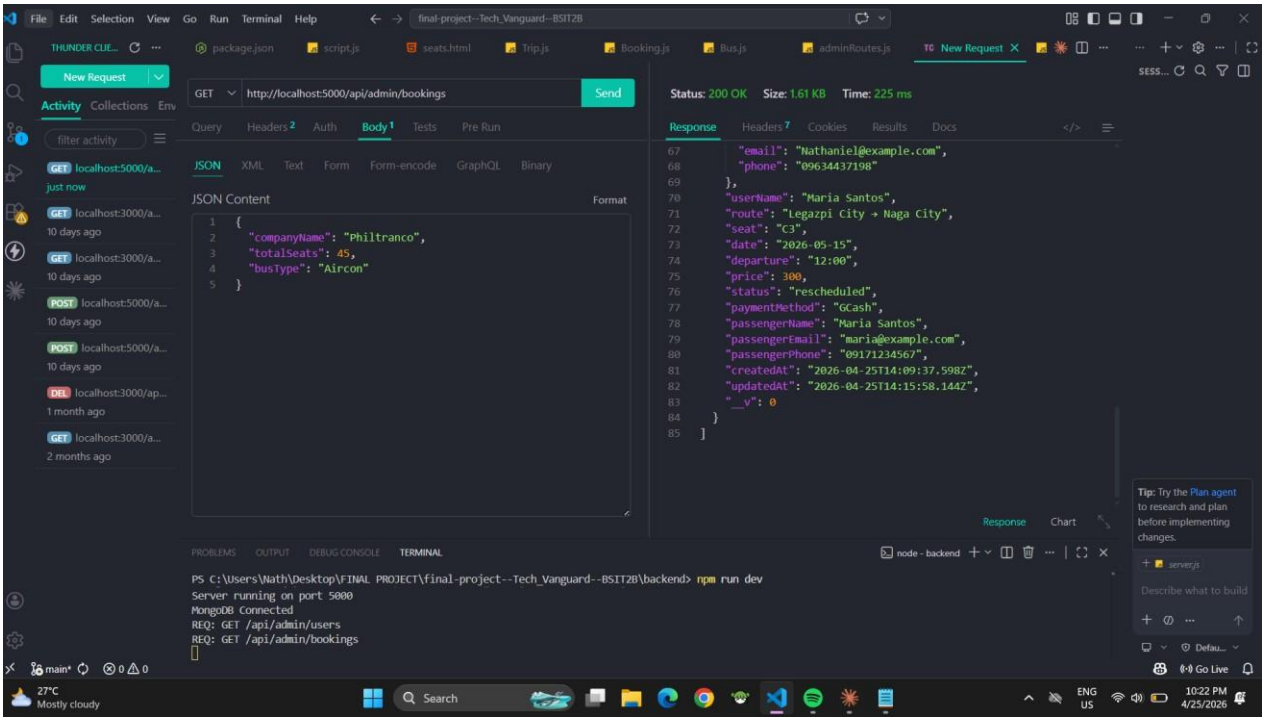
5. **Get All Buses** / GET request to retrieve all buses. It shows available buses.



6. **Admin — Get All Users**/ GET request for admin to view all users. Passwords are not included.



7. **Admin — Get All Bookings/** GET request for admin to view all bookings. It includes user details.



IV. JavaScript Code (Annotated Snippet)

Local Storage and Authentication Setup

This section shows the implementation of helper functions for managing local storage operations such as retrieving, saving, and deleting data. It also includes the authentication module, which handles user login status and retrieves current user information.

```
/* ── LOCAL STORAGE HELPERS ── */
const QR = {
  get(k, fallback = null) {
    try {
      const v = localStorage.getItem(k);
      if (v === null || v === undefined) return fallback;
      try { return JSON.parse(v); } catch { return v; }
    } catch { return fallback; }
  },
  set(k, v) {
    try { localStorage.setItem(k, typeof v === 'object' ? JSON.stringify(v) : String(v)); }
    catch(e) { console.warn('QR.set error', e); }
  },
  remove(k) { try { localStorage.removeItem(k); } catch {} },
};

/* ── PATH HELPER ── */
function toRoot(rootRelPath) {
  const p = window.location.pathname;
  let prefix = '';
  if (/\/pages\/(admin|auth|user)\//.test(p)) prefix = '../../';
  else if (/\/pages\//.test(p))               prefix = '../';
  return prefix + rootRelPath;
}

/* ── AUTH ── */
const Auth = {
  isLoggedIn() {
    const u = this.currentUser();
    return !!u && typeof u == 'object' && !!u.id;
  },
  currentUser() {
    try {
      const raw = localStorage.getItem('qr_user');
      if (!raw) return null;
      const u = JSON.parse(raw);
```

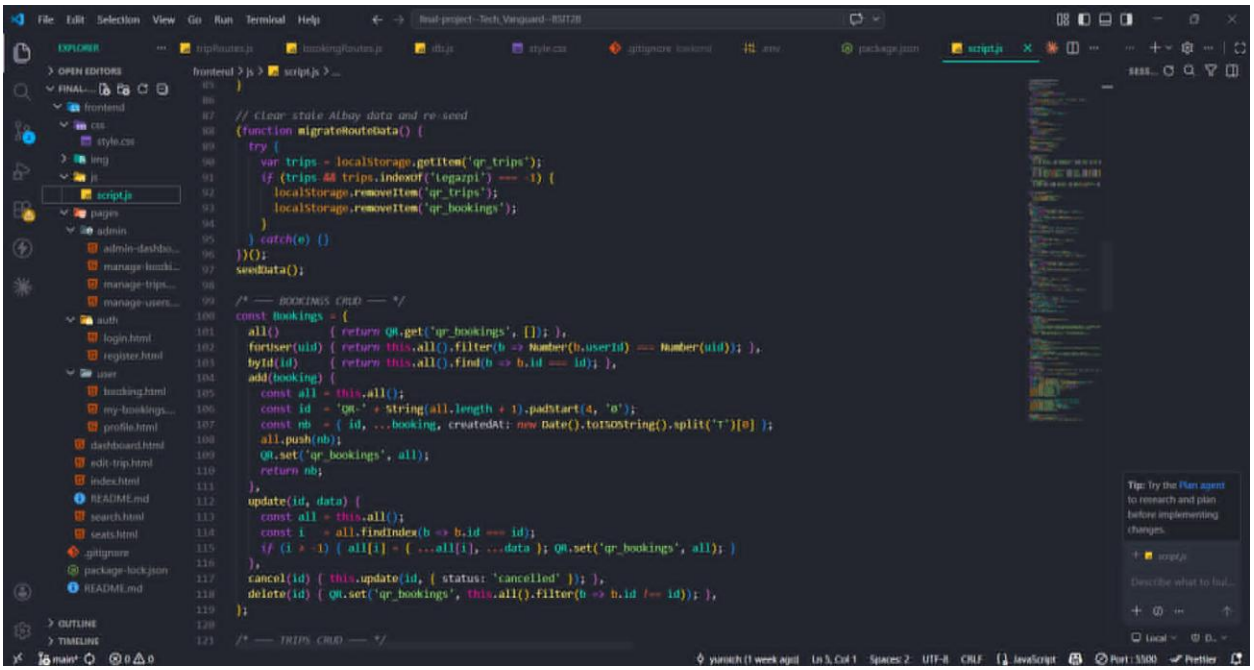
Seed Data Initialization

This part of the code initializes default data including users, trips, and bookings in local storage. It ensures that the system has sample data available for testing and demonstration purposes.

```
const Auth = {
  logout() {
    window.location.replace(toRoot('index.html'));
  },
  requireAuth() { return this.isLoggedIn(); },
  requireAdmin() {
    const u = this.currentUser();
    return !!(u && u.role == 'admin');
  },
};

/* ── SEED DEFAULT DATA ── */
function seedData() {
  if (!QR.get('qr_users')) {
    QR.set('qr_users', [
      { id: 1, name: 'Admin User',     email: 'admin@quickreserve.com', password: 'admin123', role: 'admin', phone: '090012345
      { id: 2, name: 'Maria Santos',   email: 'maria@example.com',      password: 'user123',  role: 'user',  phone: '091712345
      { id: 3, name: 'Juan dela Cruz', email: 'juan@example.com',       password: 'user123',  role: 'user',  phone: '092812345
    ]);
  }
  if (!QR.get('qr_trips')) {
    QR.set('qr_trips', [
      { id: 1, route: 'Legazpi City \u2192 Sorsogon City', duration: '1-2 hrs',    priceMin: 150, priceMax: 250, departures: [
      { id: 2, route: 'Legazpi City \u2192 Naga City',     duration: '2-3 hrs',    priceMin: 200, priceMax: 350, departures: [
      { id: 3, route: 'Naga City \u2192 Daet',             duration: '3-4 hrs',    priceMin: 300, priceMax: 450, departures: [
      { id: 4, route: 'Iriga City \u2192 Naga City',       duration: '1-2 hrs',    priceMin: 150, priceMax: 250, departures: [
      { id: 5, route: 'Ligao City \u2192 Legazpi City',    duration: '30-45 mins', priceMin: 80,  priceMax: 150, departures: [
      { id: 6, route: 'Polangui \u2192 Legazpi City',      duration: '45-60 mins', priceMin: 60,  priceMax: 120, departures: [
    ]);
  }
  if (!QR.get('qr_bookings')) {
    QR.set('qr_bookings', [
      { id: 'QR-0001', userId: 2, userName: 'Maria Santos',   route: 'Legazpi City \u2192 Naga City',     seat: 'A1', date: '2
      { id: 'QR-0002', userId: 3, userName: 'Juan dela Cruz', route: 'Legazpi City \u2192 Sorsogon City', seat: 'B3', date: '2
      { id: 'QR-0003', userId: 2, userName: 'Maria Santos',   route: 'Naga City \u2192 Daet',             seat: 'C2', date: '2
    ]);
  }
```

Bookings Management (CRUD Operations)

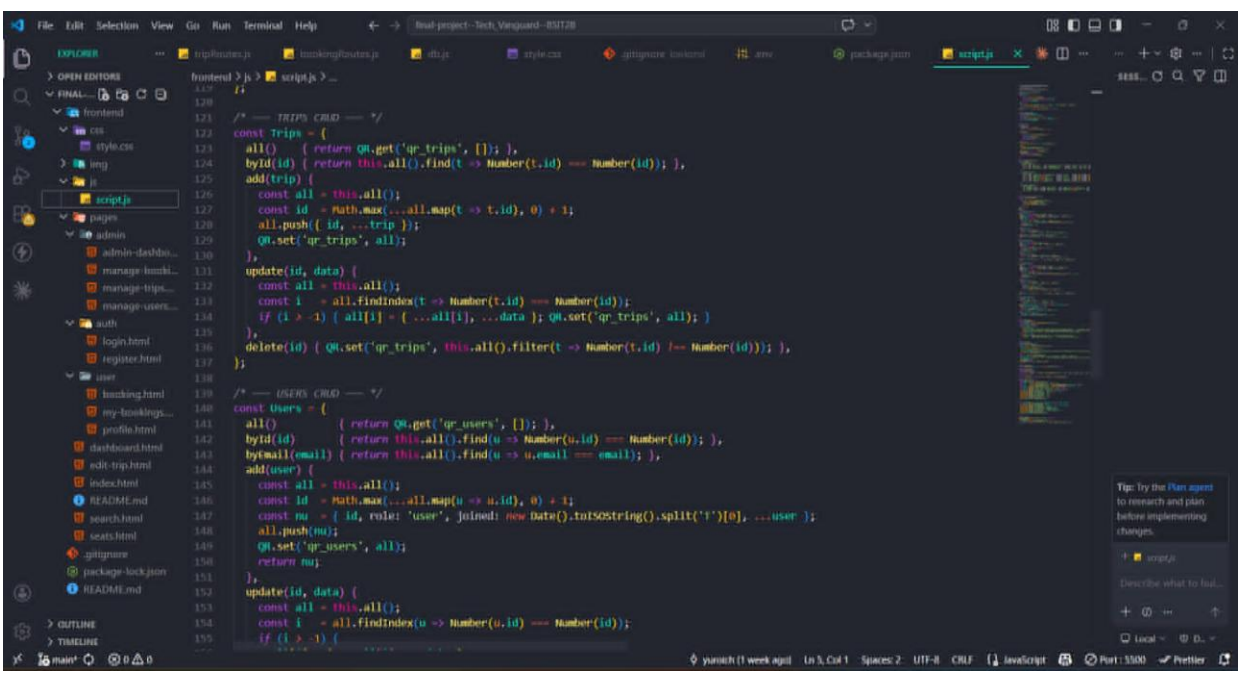
This section demonstrates the functions used to manage booking records, including adding, updating, canceling, and deleting reservations. It ensures that booking data is dynamically updated and stored correctly.



```
// Clear stale Albay data and re-seed
(function migrateRouteData() {
  try {
    var trips = localStorage.getItem('qr_trips');
    if (trips && trips.indexOf('legazpi') === -1) {
      localStorage.removeItem('qr_trips');
      localStorage.removeItem('qr_bookings');
    }
  } catch(e) {}
})();
seedData();

/* ── BOOKINGS CRUD ── */
const Bookings = {
  all()        { return QR.get('qr_bookings', []); },
  forUser(uid) { return this.all().filter(b => Number(b.userId) === Number(uid)); },
  byId(id)     { return this.all().find(b => b.id === id); },
  add(booking) {
    const all = this.all();
    const id  = 'QR-' + String(all.length + 1).padStart(4, '0');
    const nb  = { id, ...booking, createdAt: new Date().toISOString().split('T')[0] };
    all.push(nb);
    QR.set('qr_bookings', all);
    return nb;
  },
  update(id, data) {
    const all = this.all();
    const i   = all.findIndex(b => b.id === id);
    if (i > -1) { all[i] = { ...all[i], ...data }; QR.set('qr_bookings', all); }
  },
  cancel(id) { this.update(id, { status: 'cancelled' }); },
  delete(id) { QR.set('qr_bookings', this.all().filter(b => b.id !== id)); },
};

/* ── TRIPS CRUD ── */
```

Trips and Users Management

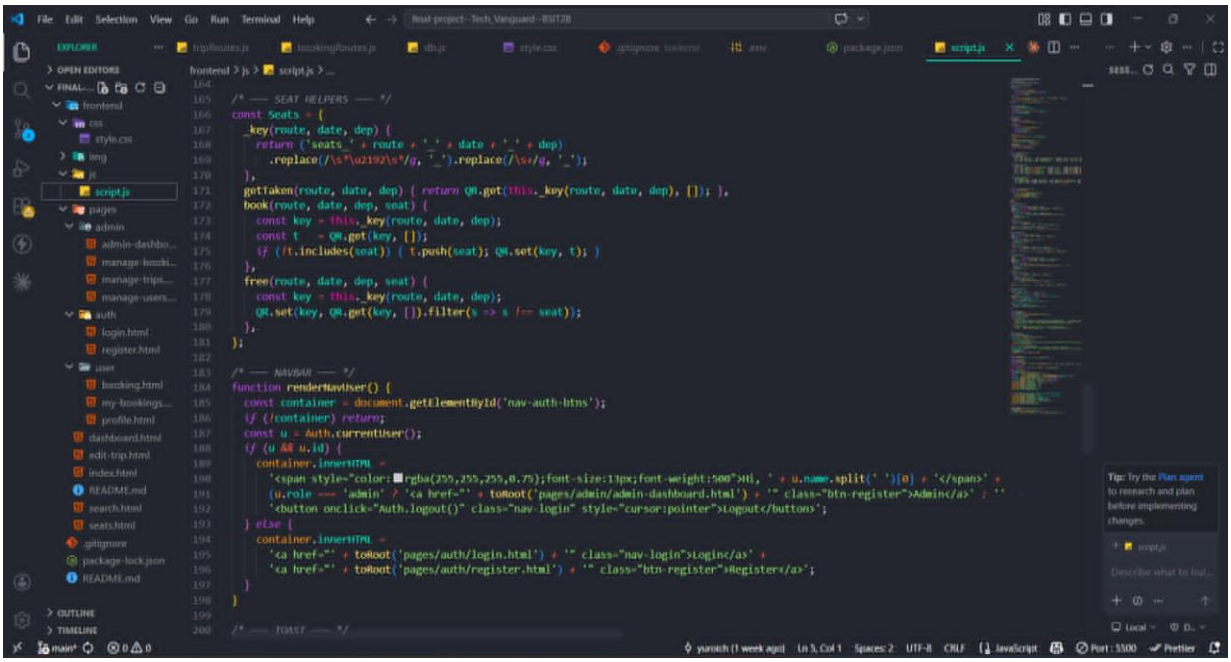
This code handles CRUD operations for both trips and users, allowing the system to manage routes, schedules, and user accounts efficiently. It ensures that all data remains accurate and up-to-date.



```
/* ── TRIPS CRUD ── */
const Trips = {
  all()    { return QR.get('qr_trips', []); },
  byId(id) { return this.all().find(t => Number(t.id) === Number(id)); },
  add(trip) {
    const all = this.all();
    const id  = Math.max(...all.map(t => t.id), 0) + 1;
    all.push({ id, ...trip });
    QR.set('qr_trips', all);
  },
  update(id, data) {
    const all = this.all();
    const i   = all.findIndex(t => Number(t.id) === Number(id));
    if (i > -1) { all[i] = { ...all[i], ...data }; QR.set('qr_trips', all); }
  },
  delete(id) { QR.set('qr_trips', this.all().filter(t => Number(t.id) !== Number(id))); },
};

/* ── USERS CRUD ── */
const Users = {
  all()          { return QR.get('qr_users', []); },
  byId(id)       { return this.all().find(u => Number(u.id) === Number(id)); },
  byEmail(email) { return this.all().find(u => u.email === email); },
  add(user) {
    const all = this.all();
    const id  = Math.max(...all.map(u => u.id), 0) + 1;
    const nu  = { id, role: 'user', joined: new Date().toISOString().split('T')[0], ...user };
    all.push(nu);
    QR.set('qr_users', all);
    return nu;
  },
  update(id, data) {
    const all = this.all();
    const i   = all.findIndex(u => Number(u.id) === Number(id));
    if (i > -1) {
```

Seat Management and Navigation Rendering

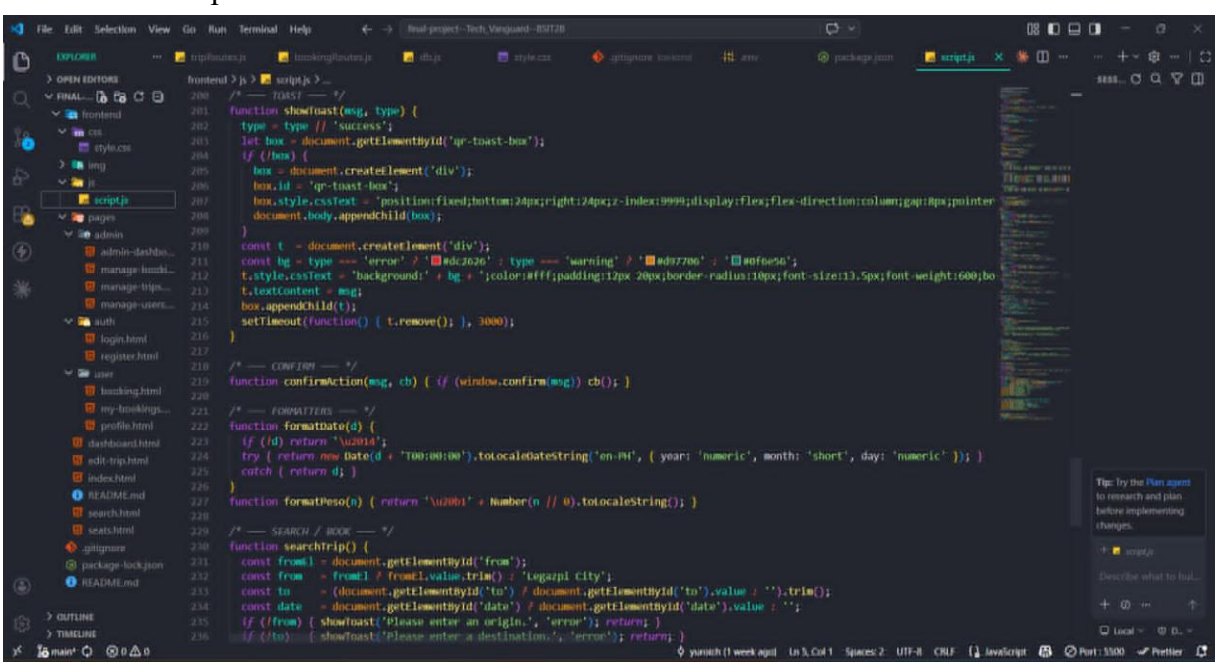
This section manages seat availability by tracking reserved and available seats based on selected routes and schedules. It also dynamically updates the navigation bar depending on the user's login status.



```
/* — SEAT HELPERS — */
const Seats = {
  _key(route, date, dep) {
    return ('seats_' + route + '_' + date + '_' + dep)
      .replace(/\s*\u2192\s*/g, '_').replace(/\s+/g, '_');
  },
  getTaken(route, date, dep) { return QM.get(this._key(route, date, dep), []); },
  book(route, date, dep, seat) {
    const key = this._key(route, date, dep);
    const t   = QM.get(key, []);
    if (!t.includes(seat)) { t.push(seat); QM.set(key, t); }
  },
  free(route, date, dep, seat) {
    const key = this._key(route, date, dep);
    QM.set(key, QM.get(key, []).filter(s => s !== seat));
  },
};

/* — NAVBAR — */
function renderNavUser() {
  const container = document.getElementById('nav-auth-btns');
  if (!container) return;
  const u = Auth.currentUser();
  if (u && u.id) {
    container.innerHTML =
      '<span style="color:rgba(255,255,255,0.75);font-size:13px;font-weight:500">Hi, ' + u.name.split(' ')[0] + '</span>' +
      (u.role === 'admin' ? '<a href="' + toRoot('pages/admin/admin-dashboard.html') + '" class="btn-register">Admin</a>' : '' +
      '<button onclick="Auth.logout()" class="nav-login" style="cursor:pointer">Logout</button>';
  } else {
    container.innerHTML =
      '<a href="' + toRoot('pages/auth/login.html') + '" class="nav-login">Login</a>' +
      '<a href="' + toRoot('pages/auth/register.html') + '" class="btn-register">Register</a>';
  }
}

/* — TOAST — */
```

UI Helpers and Search Functionality

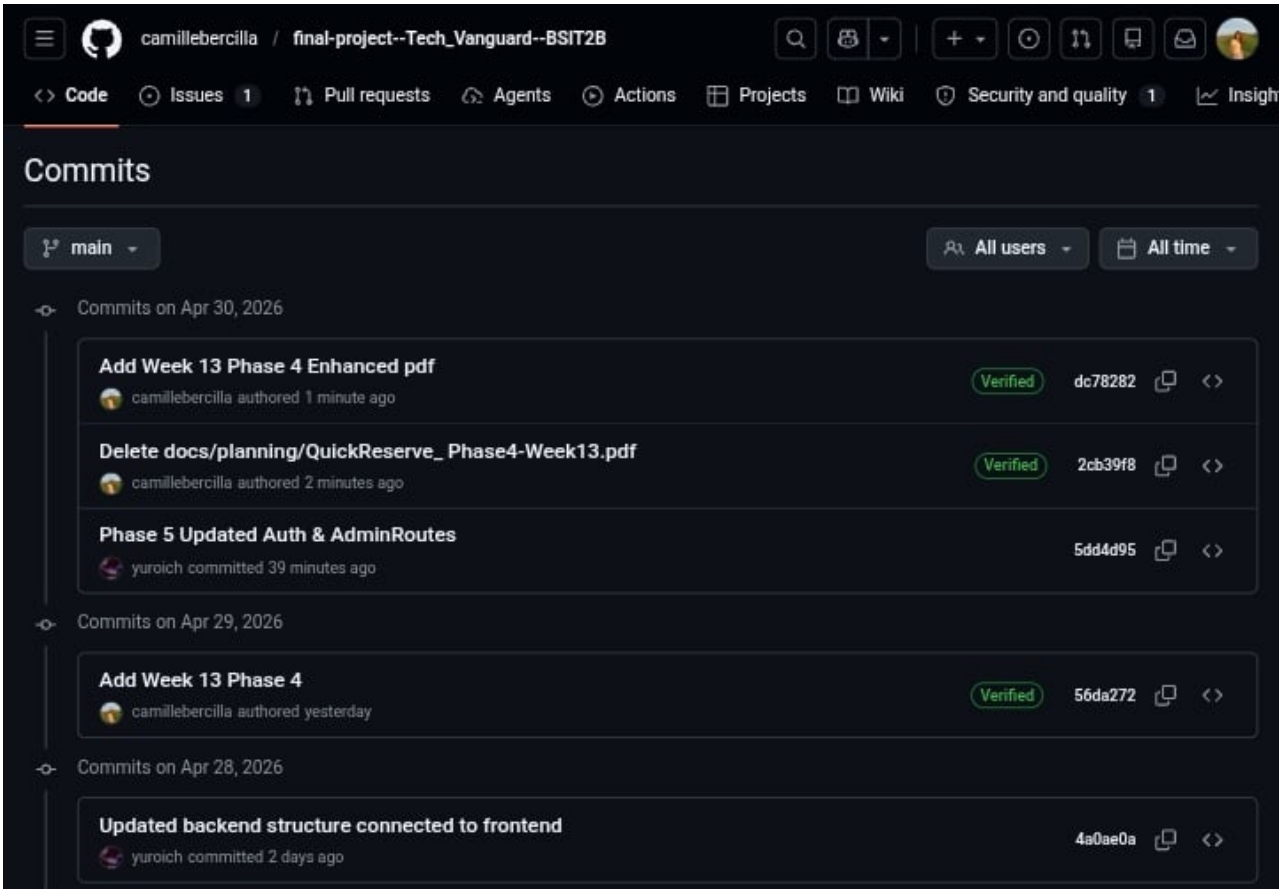
This part includes user interface helper functions such as toast notifications, confirmation prompts, and formatting utilities. It also contains the search functionality used to filter and display trips based on user input.



```
/* — TOAST — */
function showToast(msg, type) {
  type = type || 'success';
  let box = document.getElementById('qr-toast-box');
  if (!box) {
    box = document.createElement('div');
    box.id = 'qr-toast-box';
    box.style.cssText = 'position:fixed;bottom:24px;right:24px;z-index:9999;display:flex;flex-direction:column;gap:8px;pointer
    document.body.appendChild(box);
  }
  const t  = document.createElement('div');
  const bg = type === 'error' ? '#dc2626' : type === 'warning' ? '#d97706' : '#0f6e56';
  t.style.cssText = 'background:' + bg + ';color:#fff;padding:12px 20px;border-radius:10px;font-size:13.5px;font-weight:600;bo
  t.textContent = msg;
  box.appendChild(t);
  setTimeout(function() { t.remove(); }, 3000);
}

/* — CONFIRM — */
function confirmAction(msg, cb) { if (window.confirm(msg)) cb(); }

/* — FORMATTERS — */
function formatDate(d) {
  if (!d) return '\u2014';
  try { return new Date(d + 'T00:00:00').toLocaleDateString('en-PH', { year: 'numeric', month: 'short', day: 'numeric' }); }
  catch { return d; }
}
function formatPeso(n) { return '\u20b1' + Number(n || 0).toLocaleString(); }

/* — SEARCH / BOOK — */
function searchTrip() {
  const fromEl = document.getElementById('from');
  const from   = fromEl ? fromEl.value.trim() : 'Legazpi City';
  const to     = (document.getElementById('to') || document.getElementById('to').value : '').trim();
  const date   = document.getElementById('date') || document.getElementById('date').value : '';
  if (!from) { showToast('Please enter an origin.', 'error'); return; }
  if (!to)   { showToast('Please enter a destination.', 'error'); return; }
```

V. GitHub Commits Logs

GitHub commit history of the project. It highlights updates made to the frontend and backend during the development process.



VI . Individual reflections from each group member (3–5 sentences)

Each member of the group was given a specific responsibility to contribute to the successful development, testing, and documentation of the **Quick Reserve:Online Bus & Travel Booking System.**

Nathaniel O. Raynada – Project Manager /Front-End Developer

As the Frontend Developer for this phase, I built dynamic user interfaces to display data from the system. Instead of using fetch(), I worked with data stored in local storage that was connected to a MongoDB database on the backend. I used DOM manipulation to render the data into cards and structured layouts for users and products. I also made small backend adjustments to help ensure proper data flow and improve overall system integration.

Anna Mae P. Broma – Back-End Developer

As the Frontend Developer for this phase, I built dynamic user interfaces to display data from the system. Instead of using fetch(), I worked with data stored in local storage that was connected to a MongoDB database on the backend. I used DOM manipulation to render the data into cards and structured layouts for users and products. I also made small backend adjustments to help ensure proper data flow and improve overall system integration.

Camille Bercilla – GitHub Manager

I contributed to our GitHub and team workflow by helping update the frontend files and supporting README improvements to reflect new functionality. I also coordinated with the backend team for any help I would and made sure our work was organized and properly tagged for tracking. Although I didn't always push changes myself due to limited laptop access, I actively collaborated with my groupmates to ensure smooth flow of updates to the main branch.

Mariel O. Odoño – Documentation Officer

I contributed to this stage by assisting in the documentation and ensuring that the displayed data matched the system's functionality. I also helped in reviewing the frontend interface to confirm that the data was clearly presented. This experience improved my understanding of how data is retrieved and displayed in a web application.

John Edmar Bongcodin – System Tester

I worked as a tester and debugger by checking if data from the backend was properly retrieved and displayed on the frontend. I tested dynamic content, pagination, and user interface functions to ensure smooth performance. This phase helped me improve my skills in debugging and testing data flow between frontend and backend.

Justin Dasallas – Database Manager

As the Data Manager, I was responsible for managing and checking the data stored and displayed in the system. I ensured that all data used in testing and output was correct and aligned with the expected results. This experience improved my understanding of how data supports system operations.